PROGRAMACIÓN AVANZADA

Sesión 16

MergeSort

→ NOTAS DE CLASE

Índice

1 1945: ordenar 1 000 tarjetas sin memoria suficiente **2**

2 La fila del salón **2**

3 El algoritmo **3**
3.1 La función `mergesort` 3
3.2 La función `merge` 3
3.3 Paso por referencia: el contrato de la API 4

4 ¿Cuánto tarda MergeSort? **5**
4.1 Análisis informal 5
4.2 Complejidad en todos los casos 5

5 ¿Puede existir un algoritmo más rápido? **5**
5.1 El argumento de la información 5

6 Comparación con otros algoritmos **6**
6.1 ¿MergeSort es el mejor? 6

7 Ejercicios **7**

8 Resumen **8**

1945: ordenar 1 000 tarjetas sin memoria suficiente

En 1945, John von Neumann trabajaba en el diseño del EDVAC, uno de los primeros computadores electrónicos. Se enfrentó a un problema concreto: necesitaba ordenar listas de números que no cabían completas en la memoria del computador. La solución que inventó: dividir la lista en partes manejables, ordenar cada parte por separado en cinta magnética y luego *mezclar* las cintas de dos en dos.

Ese algoritmo —**MergeSort**— tiene hoy 80 años y sigue siendo la base del algoritmo de ordenamiento de Python, Java, y del ordenamiento estable de la biblioteca estándar de C++.

John von Neumann describió MergeSort en una nota interna de 1945 mientras trabajaba en el Institute for Advanced Study de Princeton. El contexto era puramente práctico: las cintas magnéticas podían leerse en secuencia pero no accederse al azar, así que el único algoritmo viable era uno que mezclara secuencias ya ordenadas.

La prueba de que MergeSort es *óptimo* vino décadas después, con la teoría de la información. Hoy sabemos que ningún algoritmo de ordenamiento basado en comparaciones puede ser asintóticamente más rápido.

Referencia: Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2.ª ed., §5.2.4. Addison-Wesley.

La fila del salón

Ocho estudiantes se paran en fila, cada uno con un número. Queremos ordenarlos.

20	31	83	92	14	42	51	63

La estrategia de MergeSort: dividir la fila por la mitad, ordenar cada mitad *por el mismo método* y luego unirlas. Aplicamos esto recursivamente hasta llegar a filas de una sola persona — que ya están ordenadas por definición.

```
[20, 31, 83, 92, 14, 42, 51, 63]
[20, 31, 83, 92]        [14, 42, 51, 63]
[20, 31]    [83, 92]    [14, 42]    [51, 63]
[20]  [31]  [83]  [92]  [14]  [42]  [51]  [63]
```

Cada `[x]` es el caso base: una persona sola ya está ordenada. A partir de ahí, subimos mezclando pares de filas ordenadas:

```
[20]  [31]  [83]  [92]  [14]  [42]  [51]  [63]
[20, 31]    [83, 92]    [14, 42]    [51, 63]
[20, 31, 83, 92]        [14, 42, 51, 63]
[14, 20, 31, 42, 51, 63, 83, 92]
```

El algoritmo

MergeSort tiene dos funciones que se llaman mutuamente:

Función	Responsabilidad
`mergesort(arr, left, right)`	Divide el subarreglo en dos mitades, ordena cada una recursivamente, y llama a `merge` para unirlas.
`merge(arr, left, mid, right)`	Recibe dos mitades ya ordenadas y las fusiona en una sola secuencia ordenada, usando un buffer temporal.

• La función `mergesort`

```
void mergesort(int* arr, int left, int right) {
    if (left >= right) return;              // caso base: 0 o 1 elemento

    int mid = (left + right) / 2;

    mergesort(arr, left,     mid);          // ordena mitad izquierda
    mergesort(arr, mid + 1, right);         // ordena mitad derecha
    merge     (arr, left, mid, right);      // fusiona las dos mitades
}
```

Listing 1: mergesort: divide y recurre

El punto medio se calcula como `(left + right) / 2`. La mitad izquierda cubre los índices `[left, mid]` y la derecha `[mid+1, right]`.

✔ Recuerda

El **caso base** es `left >= right`, no solo `left == right`. Si `left > right` el subarreglo está vacío y tampoco hay nada que hacer. El `>=` lo cubre todo de forma segura.

• La función `merge`

Fusionar dos mitades ordenadas requiere un **buffer temporal**: no podemos reubicar elementos en el arreglo original sin sobrescribir valores que todavía necesitamos leer.

```
void merge(int* arr, int left, int mid, int right) {
    int n = right - left + 1;
    vector<int> temp(n);            // buffer temporal de tamano exacto

```

```
    int i = left, j = mid + 1, k = 0;

    // Compara el frente de cada mitad y toma el menor
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    // Copia el resto de la mitad que no se agoto
    while (i <= mid)   temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    // Devuelve temp al arreglo original
    for (int k = 0; k < n; k++)
        arr[left + k] = temp[k];
}
```

Listing 2: merge: fusión de dos mitades ordenadas

Paso	Comparación	Acción	temp
1	$20 \leq 14$? No	toma 14	[14]
2	$20 \leq 42$? Sí	toma 20	[14,20]
3	$31 \leq 42$? Sí	toma 31	[14,20,31]
4	$83 \leq 42$? No	toma 42	[14,20,31,42]
5	$83 \leq 51$? No	toma 51	[14,20,31,42,51]
6	$83 \leq 63$? No	toma 63	[14,20,31,42,51,63]
7	Der. agotada	copia 83,92	[14,20,31,42,51,63,83,92]

• Paso por referencia: el contrato de la API

Las firmas de ambas funciones son un ejemplo de diseño de API mediante paso por referencia:

```
void mergesort(int* arr, int left, int right)
void merge     (int* arr, int left, int mid, int right)
```

Parámetro	Tipo	Significado
`arr`	`int*`	Referencia al arreglo — se modifica directamente
`left`	`int`	Inicio del subarreglo actual (por valor: solo se lee)
`right`	`int`	Fin del subarreglo actual (por valor: solo se lee)
`mid`	`int`	Punto de división (por valor: solo se lee)

En ningún momento se copia el arreglo. Todas las llamadas recursivas trabajan sobre el **mismo bloque de memoria**. El tipo del parámetro comunica la intención:

`int*` significa “voy a modificar esto”; `int` significa “solo lo leo”.

¿Cuánto tarda MergeSort?

• Análisis informal

El árbol de llamadas tiene $\log_2 n$ niveles. En cada nivel se procesan los n elementos exactamente una vez (repartidos entre las distintas llamadas a `merge` de ese nivel). Total:

$$T(n) = n \cdot \log_2 n \quad \Rightarrow \quad \mathcal{O}(n \log n)$$

A diferencia de Bubble Sort ($\mathcal{O}(n^2)$), MergeSort con $n = 10^6$ elementos requiere *veinte millones* de operaciones en lugar de *un billón*.

Algoritmo	$n = 100$	$n = 10^3$	$n = 10^6$	$n = 10^9$
Bubble Sort $\mathcal{O}(n^2)$	10 000	1 000 000	10^{12}	10^{18}
MergeSort $\mathcal{O}(n \log n)$	700	10 000	2×10^7	3×10^{10}

• Complejidad en todos los casos

MergeSort siempre divide por la mitad, sin importar los datos de entrada. Por eso tiene la misma complejidad en el peor, promedio y mejor caso: $\mathcal{O}(n \log n)$ en los tres.

¿Puede existir un algoritmo más rápido?

Esta es una de las preguntas más hermosas de la informática teórica, y tiene una respuesta definitiva: **no**. Ningún algoritmo de ordenamiento basado en comparaciones puede ser asintóticamente más rápido que $\mathcal{O}(n \log n)$.

• El argumento de la información

¿Cuántas formas distintas hay de ordenar n elementos? La respuesta es $n!$ (*n factorial*): para $n = 8$, son $8! = 40\,320$ posibles órdenes.

Cuando el algoritmo hace una comparación ($a \leq b$?), el resultado puede ser verdadero o falso: separa el espacio de posibilidades en **dos mitades**. Para distinguir entre $n!$ posibilidades usando preguntas de sí/no se necesitan al menos:

$$\log_2(n!) \text{ comparaciones}$$

Por la **aproximación de Stirling** ($\log_2(n!) \approx n \log_2 n - n \log_2 e$):

$$\log_2(n!) = \Theta(n \log n)$$

Cualquier algoritmo de ordenamiento por comparaciones necesita, en el peor caso, $\Omega(n \log n)$ comparaciones. MergeSort usa exactamente $\Theta(n \log n)$: **es óptimo**.

Hay algoritmos de ordenamiento más rápidos que $\mathcal{O}(n \log n)$, como **Counting Sort** o **Radix Sort**, pero no funcionan por comparaciones. Explotan propiedades específicas de los datos (por ejemplo, que los números están acotados en un

rango). Si los datos son enteros entre 0 y 1 000, Counting Sort ordena en $\mathcal{O}(n)$ — pero falla si los datos son cadenas arbitrarias. MergeSort funciona con cualquier dato que pueda compararse.

David Knuth formalizó el límite inferior $\Omega(n \log n)$ para ordenamiento por comparaciones en el Volumen 3 de *The Art of Computer Programming* (1973), usando árboles de decisión. La demostración completa usa la altura mínima de un árbol binario con al menos $n!$ hojas.

Referencia: Knuth, D. E. (1998). *TAOCP*, Vol. 3, §5.3.1.

Comparación con otros algoritmos

Algoritmo	Mejor caso	Caso medio	Peor caso	Memoria extra	Estable
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Sí
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	No
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Sí
MergeSort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	**Sí**
QuickSort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	No
HeapSort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	No

- **¿MergeSort es el mejor?**
  Depende del contexto.

- **Peor caso garantizado**: MergeSort y HeapSort son los únicos que garantizan $\mathcal{O}(n \log n)$ siempre. QuickSort puede degradarse a $\mathcal{O}(n^2)$ con entradas adversas (como un arreglo ya ordenado con pivote en el primer elemento).
- **Velocidad práctica en RAM**: QuickSort suele ser más rápido en la práctica porque accede a la memoria en patrones *cache-friendly* (localidad de referencia). MergeSort salta entre la mitad izquierda y la derecha, causando más *cache misses*.
- **Estabilidad**: MergeSort preserva el orden relativo de elementos iguales (si el alumno A y el alumno B tienen la misma calificación y A estaba antes que B, A sigue antes después de ordenar). Esto importa en bases de datos y aplicaciones donde el orden original tiene significado.
- **Ordenamiento externo**: cuando los datos no caben en RAM (archivos de terabytes), MergeSort es imbatible. Se cargan trozos, se ordenan en memoria, y se mezclan las secuencias resultantes en disco — exactamente lo que von Neumann resolvió con cintas magnéticas.

TimSort es el algoritmo que usa Python (`list.sort()`), Java (`Arrays.sort` para objetos) y la biblioteca estándar de C++ (`std::stable_sort`). Fue diseñado por

Tim Peters en 2002.

La idea: en la práctica, los arreglos reales casi nunca son completamente aleatorios — tienen "runs" (subsecuencias ya ordenadas). TimSort detecta esos runs, los extiende con Insertion Sort hasta un tamaño mínimo, y los fusiona con el algoritmo de MergeSort. Obtiene $\mathcal{O}(n)$ en el mejor caso (arreglo ya ordenado) y $\mathcal{O}(n \log n)$ en el peor.

Referencia: Peters, T. (2002). "[Python-Dev] Sorting". https://bugs.python.org/issue4255
Artículo descriptivo: https://en.wikipedia.org/wiki/Timsort

El comando `git merge` que une ramas de código tiene una lógica análoga a MergeSort: toma dos secuencias de cambios (commits) ya ordenadas por fecha y las fusiona en una sola historia coherente, resolviendo conflictos cuando los mismos archivos fueron modificados en ambas ramas. No es el mismo algoritmo, pero el patrón de "fusionar dos secuencias ordenadas" está en el corazón de cómo Git maneja la historia del código.

Ejercicios

• Ejercicio 1 — Traza manual

Aplica MergeSort al arreglo `[5, 3, 8, 1, 4, 2, 7, 6]` a mano (lápiz y papel). Dibuja el árbol de divisiones y la secuencia de fusiones, mostrando el estado del arreglo después de cada llamada a `merge`.

• Ejercicio 2 — Contar comparaciones

Modifica la función `merge` para que cuente el número de comparaciones (`arr[i] <= arr[j]`) realizadas. Luego modifica `mergesort` para acumular ese conteo y al final imprime cuántas comparaciones se hicieron en total para el arreglo de 8 elementos.
¿Coincide con la predicción teórica de $n \log_2 n = 8 \times 3 = 24$?

• Ejercicio 3 — Orden descendente

Modifica `merge` para que el resultado quede en orden **descendente**. ¿Cuántas líneas de código cambiaste? ¿Qué te dice eso sobre el diseño del algoritmo?

• Ejercicio 4 — MergeSort genérico con comparador

Modifica las firmas de `merge` y `mergesort` para que acepten un parámetro `bool(*cmp)(int, int` — un apuntador a función comparadora, igual que hicimos con Bubble Sort. Verifica que con `cmp = [](int a, int b) { return a <= b; }` el resultado es ascendente y con `a >= b` es descendente, sin cambiar el cuerpo de las funciones.

Resumen

✔ Recuerda

- MergeSort divide el arreglo hasta llegar a subarreglos de 1 elemento (**caso base**), luego los fusiona de abajo hacia arriba.
- **Complejidad**: $\mathcal{O}(n \log n)$ en todos los casos — el mejor, el promedio y el peor.
- **Memoria extra**: $\mathcal{O}(n)$ para el buffer temporal en `merge`.
- **Estable**: preserva el orden relativo de elementos iguales.
- Es **óptimo**: ningún algoritmo de comparaciones puede hacerlo mejor (prueba del límite inferior $\Omega(n \log n)$).
- QuickSort es más rápido en la práctica (cache), pero $\mathcal{O}(n^2)$ en el peor caso. MergeSort garantiza $\mathcal{O}(n \log n)$ siempre.
- TimSort (Python, Java, `std::stable_sort`) es MergeSort adaptado para arreglos reales con runs ya ordenados.
- `int* arr` en la firma significa *paso por referencia*: la función modifica el arreglo original sin copiarlo.